

Assignment 2

Benjamin Labrecque

March 25, 2026

Exercise 2.1

Execution times for running and incrementing / decrementing a counter with a varying number of threads. It should be noted that the **Counter** program does not finish with the counter set to 0, unlike the other programs, as it does not make use of any synchronization constructs.

Program	Threads	Duration (s)
Counter	2	0.3906
Counter	4	0.5539
Counter	8	1.8143
Counter	16	3.5781
CounterAtomic	2	0.3995
CounterAtomic	4	0.9018
CounterAtomic	8	2.8182
CounterAtomic	16	5.7090
CounterBakery	2	1.4331
CounterBakery	4	4.6810
CounterBakery	8	45.1150
CounterFilter	2	1.4120
CounterFilter	4	5.8230
CounterFilter	8	73.6610
CounterLock	2	1.4290
CounterLock	4	0.7995
CounterLock	8	1.5744
CounterLock	16	3.1960
CounterMonitor	2	1.7864
CounterMonitor	4	4.6900
CounterMonitor	8	9.6430
CounterMonitor	16	10.3840

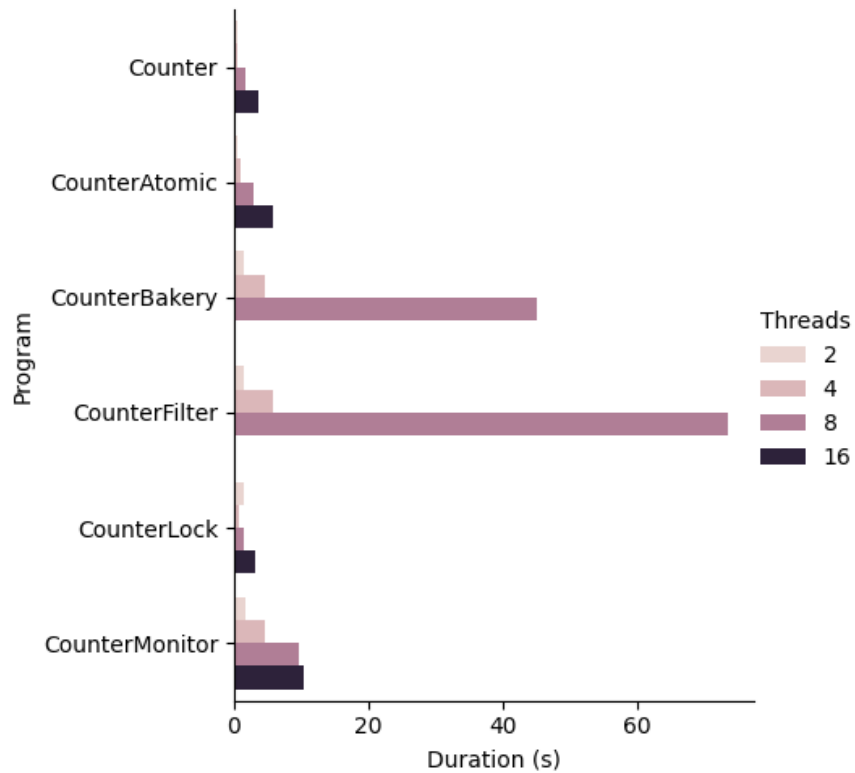


Figure 1: Execution times for incrementing / decrementing an integer 10'000'000 times. **CounterBakery** and **CounterFilter** scale poorly. Their execution time with 16 threads was not recorded as it would take >15 minutes.

Exercise 2.2.1 and 2.2.2

Implemented a basic read-write lock which avoids writer starvation. See ReadWriteLock.java

Exercise 2.2.3

Implemented a read-write lock that respects FIFO order. It is however not reentrant. See ReadWriteLockFIFO.java

For the following exercises I drew an arrow diagram to determine if a linearizable solution could be found.

Exercise 2.3.1.a

Linearizable: no

Sequentially consistent: yes [A:w1, B:r1, A:w2, C:r2, A:w1, C:r1]

A linearizable history:

A: r.write(1)

A: r:void

A: r.write(2)

A: r:void

C: r.read()

C: r:2

A: r.write(1)

A: r:void

C: r.read()

C: r:1

B: r.read()

B: r:1

Exercise 2.3.1.b

Linearizable: yes

Sequentially consistent: yes

Exercise 2.3.2.a

Linearizable: yes

Sequentially consistent: yes

Exercise 2.3.2.b

Linearizable: no

Sequentially consistent: no

A linearizable, and therefore also sequentially consistent, history:

A: s.push(10)

A: s:void

B: s.push(10)

B: s:void

A: s.pop()

A: s:10

A: s.pop()

A: s:10

B: s.empty()

B: s:true

Exercise 2.3.3.a

Linearizable: no

Sequentially consistent: no

This sequence is illegal, 'y' is enqueued only once but dequeued twice.